

这个系统 由什么组成， 用什么造的

一份写给非本项目成员看的速览：AgentLoom 有哪几个真正干活的模块，各自解决什么问题，背后分别押了哪些技术。不讲代码细节，只讲结构与理由。

11

代码包

41

后端业务模块

5

编程语言

01 · 它是什么

像画流程图一样， 把一群 AI 组装成生产线

AgentLoom 是一个**多智能体 workflow 编排平台**。用户在网页画布上拖出若干节点、连上线，就得到一条 AI 工作流；点“运行”，后端按图的先后顺序把每个节点真正执行掉，过程实时回传到页面上。

■ Studio

网页端。画布、Agent 配置、监控与管理页面都在这里。相当于**驾驶舱**。

■ Server

后端。存数据、排队、调度节点、跟大模型对话、推实时事件。相当于**大脑与总调度**。

■ Sandbox

隔离的微型虚拟机。AI 真正跑命令、写文件的地方。相当于**带门禁的车间**。

系统分层

用户端	STUDIO (WEB) · FLUTTER (MOBILE) · 编辑器经 ACP 接入
接入层	REST /API/V1 · SOCKET.IO 实时通道 · API KEY / JWT
业务模块层	工作流 · 智能体 · 沙箱 · 插件 · 治理 (41 个模块)
运行时层	BULLMQ 队列 WORKER · FIRECRACKER MICROVM · WASM 插件
基础设施	POSTGRESQL · REDIS · QDRANT · MINIO

一个关键区分

工作流和**智能体**是两个平级的顶层概念，各有自己的定义、版本、执行记录；智能体也可以被塞进工作流里当一个节点用。不是“工作流里才有 AI”。



02 · 核心模块 A

workflow编排：把图变成执行

画布上的图是一份数据，不是程序。真正难的是：怎么保证用户连的线是合法的，以及怎么让后端按图跑对顺序、跑挂了能说清挂在哪。

■ 连线前：类型对不上就不让连

每个节点的输入输出都有类型（文本、模型、工具、沙箱、记忆...共 14 种）。就像插头和插座，插不上直接拒绝，不用等到运行时才报错。这套兼容性判断用 **Rust 写成、编译为 WASM** 在浏览器里跑，拖动时实时给反馈。

■ 运行时：按拓扑顺序推进

后端的节点调度器沿着图的依赖关系一个个往下推，支持条件分支、循环、子流程嵌套。每步的状态和产物都落库，断线重连能把之前的事件**增量补播**回来，页面不会“失忆”。

触发一条工作流的四种方式

01 手动运行 画布里点 Run，跑的是当前草稿	02 定时任务 cron 表达式，到点自动跑	03 Webhook 外部系统回调，带签名校验	04 API 事件 如 GitHub 事件，匹配后触发
--	--	---	---

草稿与发布，是两件事

编辑器里随手保存的是**草稿快照**，只有点“发布”才会生成一个带版本号的正式快照。外部触发、移动端、API 调用一律跑已发布版本；只有 Studio 里的 Run 才跑草稿。这样“边改边有人在用”不会互相打断。

用到的技术

- 画布：React Flow (@xyflow/react 12)
- 类型引擎：Rust + wasm-bindgen
- 调度：NestJS 模块 + BullMQ 队列
- 实时：Socket.IO 双命名空间

B

03 · 核心模块 B

智能体：有记忆、会用工具、能改自己

这里的 Agent 不是“一次问答”，而是一个有持久身份的东西：它有版本、有对话历史、有长期记忆、有一套被授权的工具，还能提议修改自己的配置。

■ 记忆

对话里值得记住的结论会存成**知识图谱**，下次按语义召回。向量检索交给 Qdrant，页面上还能把图谱画出来看（d3-force + dagre 布局）。

■ 技能（Skill）

一份 Markdown 写的操作手册，存在对象存储里。运行时按需把可用技能清单注入系统提示词，用得上再取正文——相当于**随身带的工具书**，不占平时的上下文。

■ 工具权限

写文件、改配置这类有后果的动作会**停下来问你**；你可以本次批准，也可以在这轮会话里记住选择。等待期间任务不算失败，只是挂起。

■ 自进化

Agent 能查看自己的状态、提出配置变更、经审批后应用。写操作一律走审批，不给它“悄悄改自己”的口子。

模型不是写死的

智能路由按策略挑模型：省 token、省钱、优先质量、优先延迟、历史表现最好，或者**回退链**（这个模型挂了自动换下一个重试）。默认走回退链，因为它对线上最友好。

两种运行形态

形态	跑在哪	能不能执行代码
sandbox	微型虚拟机内	可以，含终端与文件
no_sandbox	后端进程内	不行，只有非代码工具

两者都支持技能、知识库、记忆与自进化；差别只在“能不能真的动手操作机器”。



04 · 核心模块 C

沙箱：让 AI 动手，但只能在笼子里

一旦允许 AI 执行命令、装依赖、改文件，问题就从“回答得对不对”变成“它会不会把机器搞坏、会不会读到别人的数据”。这块是整个系统里安全要求最高的部分。

■ 为什么不用容器

容器共享宿主内核，跑不可信的 AI 生成代码风险偏大。这里用 **Firecracker 微型虚拟机**：每个沙箱是独立内核的轻量 VM，毫秒级启动，隔离强度接近传统虚拟机，开销接近容器。

笼子是怎么焊住的

- **网络身份绑定**：网卡入口同时校验分配的 IP 与 MAC，冒充别人的地址直接丢包；关掉 IPv6，默认拒绝访问内网与其他沙箱。
- **文件边界**：所有路径必须落在工作区内，解析软链接后再判一次，防目录穿越。
- **资源上限**：CPU、内存、终端并发数、单次输出大小、生命周期超时全部硬限制，超了就杀。
- **证书不对称**：沙箱内不持有应用侧证书，回调必须经受控中继回到发起它的那个进程。

顺带打通了外部编辑器

系统实现了 **ACP (Agent Client Protocol)**：支持该协议的编辑器可以通过标准输入输出直接把 AgentLoom 的 Agent 接进来用，共享同一套文件读写与终端能力，并且写入前一定弹权限确认。能力是协商出来的——对端不支持终端，就不暴露终端。

用到的技术

- 虚拟机管理：Go + Firecracker SDK
- 沙箱内代理：Go guestd + Fastify
- 网络：CNI + tc + nftables
- 协议：JSON-RPC 2.0 over stdio

D

05 · 核心模块 D

平台面：多租户、可审计、可扩展

前面三块决定“能不能跑”，这一块决定“敢不敢给别人用”。多个组织共用一套系统，数据不能串，行为要留痕，资源要有闸门。

■ 数据隔离

每个请求进来先认出它属于哪个组织，之后所有数据库操作都在带租户上下文的事务里跑，数据库层再叠一层行级安全策略。**两道锁，不靠应用代码自觉。**

■ 端到端加密

模型输出等敏感内容用“非对称密钥包对称密钥”的方式加密后入库，私钥只存在用户浏览器本地。**数据库被整体拖走也读不出内容**——但要诚实说明：它防的是静态数据泄露，不防在线进程被攻陷。

■ 审计与配额

关键操作写只追加的审计表，旧数据自动归档，查询时热表与归档表合并回放。配额侧管住并发数、日调用量、每分钟限流，超限直接在入口拒绝并留痕。

■ 插件生态

第三方节点以 **WASM** 形式上传，装包要验 RSA 签名，执行在独立沙箱里且有固定的时间与内存上限。配套市场、用量记录与收益分账。

通知

站内 / 邮件 / 推送三通道，执行完成、失败、需人工介入都会 fan-out。

分享与市场

短链分享 workflow 与 Agent，或上架市场供他人安装，导入时会记录来源。

监控面板

按 15 分钟 / 1 小时 / 24 小时窗口看执行、治理、通知、队列快照。

私有部署

组织级 SMTP、模型代理、证书与许可证配置，配 Compose 与 Helm 资产。

06 · 技术栈

各层押了什么

层	技术	为什么是它
网页端	React 19 · Vite 7 · TypeScript	生态最厚，画布类库都在这一侧
画布	React Flow 12	节点连线、缩放、自定义节点开箱可用
前端状态	TanStack Router / Query · Zustand	服务端数据交给 Query 缓存，本地态才用 Zustand
样式	Tailwind 4 · Radix · CVA	无样式组件保证可访问性，样式自己控制
后端框架	NestJS 11 + Fastify 5	模块化依赖注入；HTTP 层选 Fastify 而非 Express，吞吐更高
数据库访问	Drizzle ORM	类型直接从表结构推导，无装饰器与运行时魔法
校验	Zod 4	一份 schema 同时产出类型、校验与接口文档
队列	BullMQ (Redis)	长任务、周期任务、重试都靠它，Web 进程不被拖死
实时	Socket.IO 4	执行进度与对话流式输出，断线可增量补播
模型接入	Vercel AI SDK	统一多家模型的流式接口
存储	PostgreSQL · Redis · Qdrant · MinIO	关系数据 / 队列缓存 / 向量检索 / 对象文件，各司其职
沙箱	Go 1.25 · Firecracker · CNF	贴近系统层的活交给 Go，隔离交给微型虚拟机
类型引擎	Rust + WASM	递归结构比较用枚举与模式匹配表达最清晰，产物小、可在 Web Worker 里跑不阻塞画布。 当初想要的「两端共用一份规则」至今只有前端接入
插件运行	Extism (WASM)	跑第三方代码而不给它宿主权限
移动端	Flutter · Riverpod · go_router · Dio	一套代码覆盖双平台，复用同一套接口契约
测试	Vitest 4 · Testcontainers	与 Vite 同源；集成测试起真实 PostgreSQL 容器
工程	pnpm workspace · ESLint · Prettier	七个包共用一份锁文件与统一版本目录
部署	Docker Compose · Helm · Nginx	开发环境一键起，私有化交付有 Helm 包

一条贯穿全项目的约定

跨端传输的数据结构**只允许有一个来源**：契约包定义 schema，接口类型由后端导出的 OpenAPI 自动生成。网页、移动端、后端不许各自手抄一份 —— 手抄就会漂移，漂移就会在半年后变成线上事故。

07 · 串起来看

点一次“运行”，发生了什么

把前面的模块按时间顺序排一遍，就是这条链。每一步失败都有明确归属，不会出现“它挂了但不知道挂在哪”。

01 入口鉴权 认出组织身份，套上租户上下文	02 配额准入 并发、日用量、治理暂停状态先过闸	03 落库建执行 写执行记录，事务提交后才入队	04 队列派发 Worker 领任务，Web 进程不阻塞
05 节点调度 按依赖推进，分支与循环在此展开	06 动手干活 调模型、查知识库、进沙箱执行命令	07 实时回传 每步事件推到页面，带序号可补播	08 收尾 加密敏感产物、写审计、发通知、回收沙箱

要人工点头的地方

第 6 步里，Agent 想写文件或改配置时会挂起并请求授权；超时怎么办由策略决定（自动同意 / 拒绝 / 升级给上级），升级最多三次。

刷新页面不丢进度

第 7 步的事件带递增编号，前端重连时报上“我收到第几号了”，服务端只补后面的增量，不重放全部。

沙箱不一定马上销毁

会话型用完即毁；持久型只停机、保留磁盘，下次可重启接着用，工作区内容会回写成快照。

08 · 为什么这样选

四个不那么显然的决定

01 用微型虚拟机，而不是容器

跑的是模型现场生成的代码，等于把不可信输入直接交给机器执行。容器共享宿主内核，一个内核漏洞就穿透了。代价是需要自己管虚拟机生命周期、网络和磁盘，比 `docker run` 复杂一个量级——这部分复杂度换来的是隔离边界，值得。

02 把端口兼容性判断写成 Rust/WASM

用户拖线时需要即时反馈「这两个能不能接」。判定逻辑是递归的结构比较，用 Rust 的枚举加模式匹配表达最清晰，编译成 WASM 后放进 Web Worker 跑，不阻塞画布。但要诚实：当初选它的另一半理由是「服务端复用同一份规则」，这一半至今没兑现——目前只有 Studio 接入，服务端没有任何调用点。按现状看这个选择偏重了，收益只剩表达力与产物体积。

03 草稿与发布分离，触发源决定跑哪份

最直觉的做法是“保存即生效”，但那意味着有人正在被这条流程服务时，另一个人的一次误保存会立刻打断线上行为。所以外部触发一律跑发布快照，只有编辑器内的手动运行跑草稿。多一层概念，换来“改和用互不干扰”。

04 宁可让功能显式失败，也不让它假装能用

典型例子是配置优化建议：分析能生成建议，但写入位置对执行不产生实际影响，于是采纳按钮在前后端都用同一份空白名单直接禁用并给出说明，只保留“忽略”。能看见的坏掉，比看不见的没生效好。

65

数据库表结构文件

14

端口数据类型

5

语言：TS / RUST / GO / DART / SQL

这个系统里，哪一部分是「我」

这套代码的绝大部分由 AI 生成。与其含糊过去，不如把边界划清楚：**设计全部是我**的，**代码按我的介入方式分三档**。划清楚之后，能被追问到什么深度也就一目了然。

我主导设计与验收	跨端契约 · 沙箱隔离边界 · 多租户隔离
我定语义、按行为验收	调度语义 · 事件协议 · 路由策略 · 类型引擎（意图未完全兑现）
生成为主、抽查为辅	业务模块 CRUD · STUDIO 页面 · 移动端 · 市场与分账

分档依据不是「写了多少行」，而是**失误的影响范围**。第一档三处的共同点：它们的错误不局限在某个模块内，而是跨语言、跨端、跨信任边界地扩散——属于**系统级边界**。所以这三处的不变量必须由我定义，并以专项测试作为验收条件。

■ 为什么这样讲比宣称手写更强

面试官验证归属只有一个手段：**顺着因果往下追问**。宣称手写，一句「agent-execution 的 worker 怎么做 turn 归一化」就会连带毁掉那三块你真的讲得清的部分。反过来，说清「哪些边界是我定的、验收标准是什么」，是更难伪造也更容易兑现的判断力。

■ 一句可直接用的表述

「系统设计是我做的，代码大量由 AI 生成。有三处的边界是我主导设计并定验收标准的——跨端契约层、沙箱隔离边界、多租户隔离——因为这三处的失误会跨端或跨信任边界扩散，不是单模块问题。这几处的实现与测试我能讲清；具体业务模块的实现细节我会答不上来。」

这句话的兑现成本

说了「这几处我能讲清」，就必须真的讲得清——面试前把这三处的实现与专项测试吃透一遍，这是可控的准备量。而**生成为主那一档的实现细节，提前承认答不上来，比现场被问穿代价小得多。**

10 · 第一档细说

三处系统级边界，各自在防什么

01 跨端契约层（Zod + OpenAPI 生成）

在防：网页、移动端、服务端各自手抄一份数据结构，抄完开始漂移。**做法：**schema 只有一份，接口类型由服务端导出的 OpenAPI 自动生成，三端一律消费生成产物。**验收方式是编译本身**—— schema 与序列化器对不上，构建就失败，这比「写个测试查一致性」强一档，因为测试可以被跳过、构建不能。

两个非直觉的坑：其一，响应 schema 不能复用带 `.default()` 的请求 schema——有默认值意味着「输入可省、输出恒存」，而 OpenAPI 按输入语义会把这些字段标成非必填，等于凭空放宽了真实传输契约，所以响应侧要单独按输出形状建一份。其二，画布的 `nodes / edges / viewport` 必须从生成类型里摘掉手写的——OpenAPI 3.0 表达不了 React Flow 的动态 data 字典与坐标元组，生成产物在这三处会退化成空对象。

可复述的因果：自动生成的价值在于「不可能漂移」；因此凡是生成会失真的位置，必须显式标出并手写，而不是默默接受一个更宽松的类型。另有一条**机械同步测试**直接读各端源码文本提取端口类型取值集合，断言各端 $\subseteq$ 全集且并集 = 全集——曾经真的漂移过（一端 12 值、一端 14 值、文档写 10 值）。

02 沙箱隔离边界

在防：模型现场生成的代码逃出笼子，或读到别的租户的东西。**文件边界（在沙箱内的 Go 进程里，不在服务端）：**只有 `/workspace` 与 `/tmp` 两个固定可写根；路径先规范化、判定在根内，再解开软链接、**解开后再判一次**。写文件另走一条更严的分支：不许把根本身当文件写，已存在的目标必须是常规文件（拒绝目录、管道、设备节点），不存在的目标则解析其父目录并确认父目录是目录且在根内。解 tar 包再单独一条：拒绝对绝对路径条目，拼接后必须仍在目标目录下。**网络边界：**虚拟机网卡入口一条白名单链，IP 包必须源 IP 与源 MAC 同时匹配、ARP 必须 MAC 与声明 IP 同时匹配、IPv6 全丢，末尾**匹配一切即丢弃兜底**。**凭据边界：**沙箱内不持有应用侧证书，回调必须经受控中继续回发起它的那个进程。

可复述的因果：容器共享宿主内核，一个内核漏洞就可能穿透。跑不可信生成代码时，隔离强度优先于启动开销——这是选微型虚拟机的核心理由，代价是要自己管理生命周期、网络与磁盘。读写路径会在规范化前后守住固定根边界；tar 解包单独校验条目路径及软链接父目录；网络用身份白名单与末尾默认丢弃。共同原则是**默认拒绝、校验后放行**，不是四处都用同一种检查。

03 多租户数据隔离

在防：某个查询漏写一个 `where organization_id = ?`，就把别的组织的数据返回出去。**实际顺序：**Middleware 先解析请求里的租户提示，但此时 JWT 尚未验签，不能把 claim 当可信身份；随后 AuthGuard 验证签名并把可信的 `tenantId` 写入 `request.user`，TenantGuard 再做租户准入； Guards 通过后，TenantTransactionInterceptor 才读取可信身份，把 handler 的数据库操作包进事务并设置 RLS 上下文。数据库表上的行级安全策略是最后一道锁。

可复述的因果：Middleware 只能做早期解析，**认证必须先于数据库租户上下文**，否则攻击者可伪造未验签 claim 影响 RLS。应用层负责建立可信上下文，数据库 RLS 假设应用查询仍可能漏条件——两层防的是不同错误。

为什么类型引擎不在这一档

它当初也是按系统级边界设计的，但核实后发现设计意图并未兑现：服务端没有接入，Rust 侧那份更严格的连线判定没有 WASM 绑定、浏览器不可达，实际生效的是 TS 侧一份略宽松的规则。**验收没抓到这个差距，所以它现在的诚实定位是探索性实现加已知技术债**，不是我能拿来背书的闭环边界。

11 · 诚实清单

已核实的现存缺口

下面每条都在写这一页时重新核对过源码。声称某块归自己之前，先确认它不在这张表上——否则一被追问演示就穿。

缺口	现状
类型引擎服务端接入	服务端 没有任何调用点 ，WASM 只在 Studio 的 Worker 里加载。此外 Rust 侧那个更严格的连线判定没有 WASM 绑定，浏览器不可达，实际生效的是 TS 侧一份略宽松的规则
对话侧断线补播	工作流侧已接通（订阅时上报游标、按 ACK 与快照推进）； 对话侧的游标字段只有声明与初始化，订阅时不上报、事件不推进 ，等于回放契约未接通
流式适配器	createVercelStreamFn 只有定义与单元测试， 全服务端无生产调用点
插件 CLI 发布	publish 只做签名并写回本地归档， 不上传 ；但命令描述写的是「发布到 Marketplace」
审计写入口	治理模块另有一条独立事务直写审计表的路径。 这是有意的 ——主事务会因阻断而回滚，审计必须独立落地，否则「拦了但没留痕」；失真的是文档措辞「唯一正式写入口」
优化建议采纳	四类建议写入的位置对执行不产生实际影响，因此前后端用同一份空白名单显式禁用采纳、只留「忽略」。 这是刻意的 fail-closed，不是残缺

两条方法纪律

一、别拿自己的旧笔记当现状。这张表最初有一条已经过时：某个面板被记作「无生产挂载点」，而当前源码里它已被正式引入并渲染。

二、别拿测试替身当生产实现。本册初稿讲文件边界时引的是一个只在环境变量开启时才注入的测试用 runtime，真正的生产实现在沙箱内的 Go 进程里，而且更严。**确认一段代码是否在生产路径上，比读懂它更重要**——讲之前逐条回源码核对，成本几分钟，收益是不在面试现场说出一句已经不成立的话。

收束

一句话记住这个系统

画布负责表达意图，调度器负责把意图变成有序执行，沙箱负责让执行有边界，平台层负责让这一切敢被多个组织同时使用。

想再往深处看

- 各包根目录的 `AGENTS.md`：模块级事实描述
- `agentloom-docs/`：中英双语文档站
- 同目录的《系统设计与开放问题》《实现细节参考册》

这份文档的边界

只讲结构与选型理由，不含接口签名、表字段与实现代码。数字与缺口清单为撰写时的仓库实测值，会随代码演进变化，引用前请回源码核对。